

Homework 3: Neural Network Tasks

Question 1: Digit Classification with MNIST

Part A: using NN

Figure 1 below shows the model architecture used for the NN model used for digit classification of the MNIST dataset. After loading the dataset, each image is flattened into a 784-dimensional vector. The NN model has 5 hidden layers which use ReLU activations along with batch normalization and dropout to improve generalization and reduce overfitting. Model summary also shows the architecture (right side of the image).

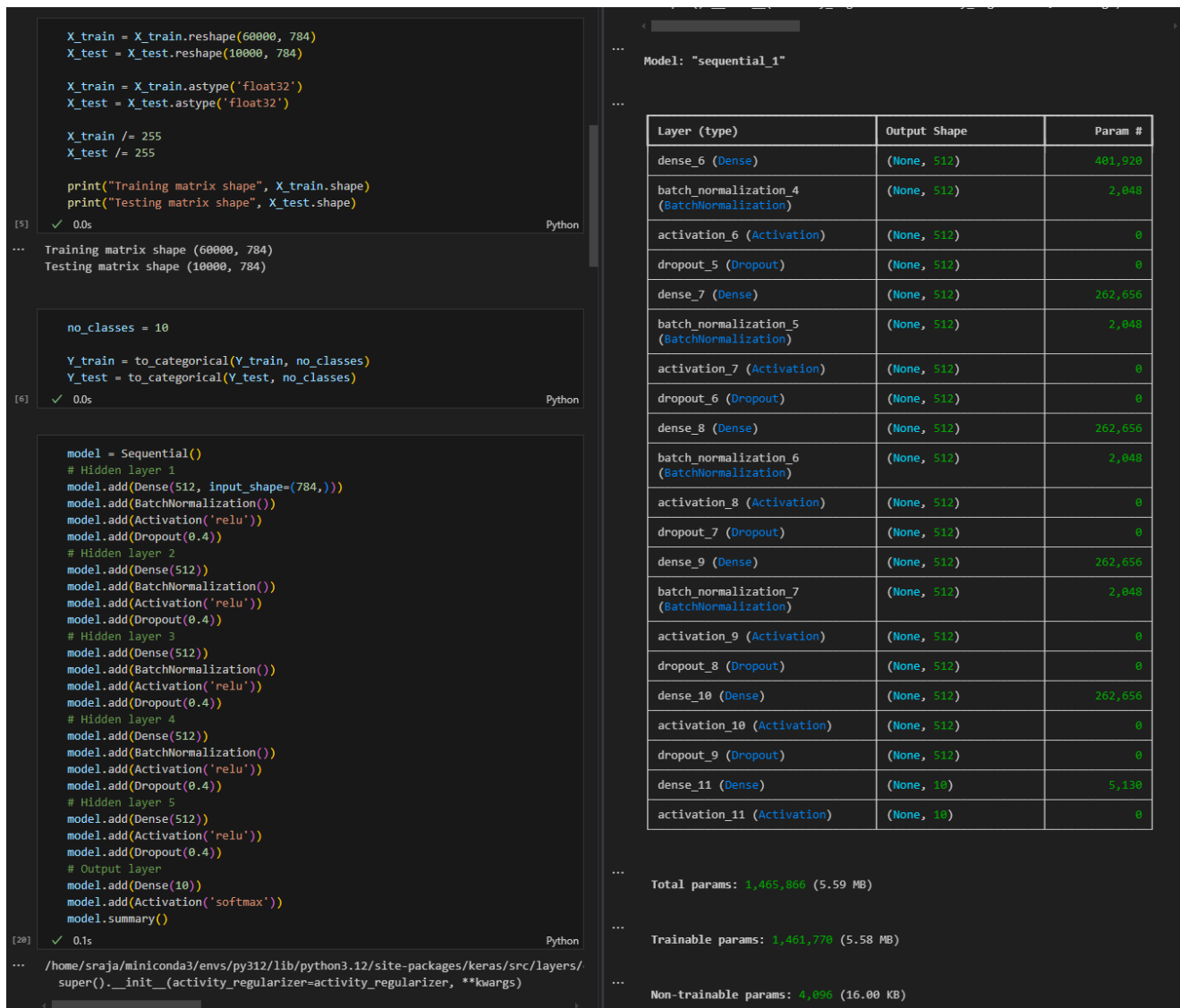


Figure 1: NN Model architecture for Digit Classification with MNIST

Figure 2 below shows the model configuration used for training. The model uses a batch size of 512 and Adam optimizer with learning rate of 1e-3 (0.001), also the model uses callbacks early stopping and model checkpoint. Figure 3 shows the accuracy and loss reported during training and test. The model achieves an accuracy of **98.26%** on the test dataset. Finally figure 4 shows the plots for training and validation accuracy and loss over epochs (left side) and the confusion matrix (right side).

```
checkpoint_cb = keras.callbacks.ModelCheckpoint('model-nn.keras', save_best_only=True)
early_stopping_cb = keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True)
ss=1e-3
batchSize = 512
optimizer=keras.optimizers.Adam(learning_rate=ss)

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

history = model.fit(X_train, Y_train, batch_size=batchSize,
                    epochs=100, verbose=1, validation_split=0.2,
                    callbacks=[checkpoint_cb, early_stopping_cb])
```

Figure 2: NN model configuration used for training

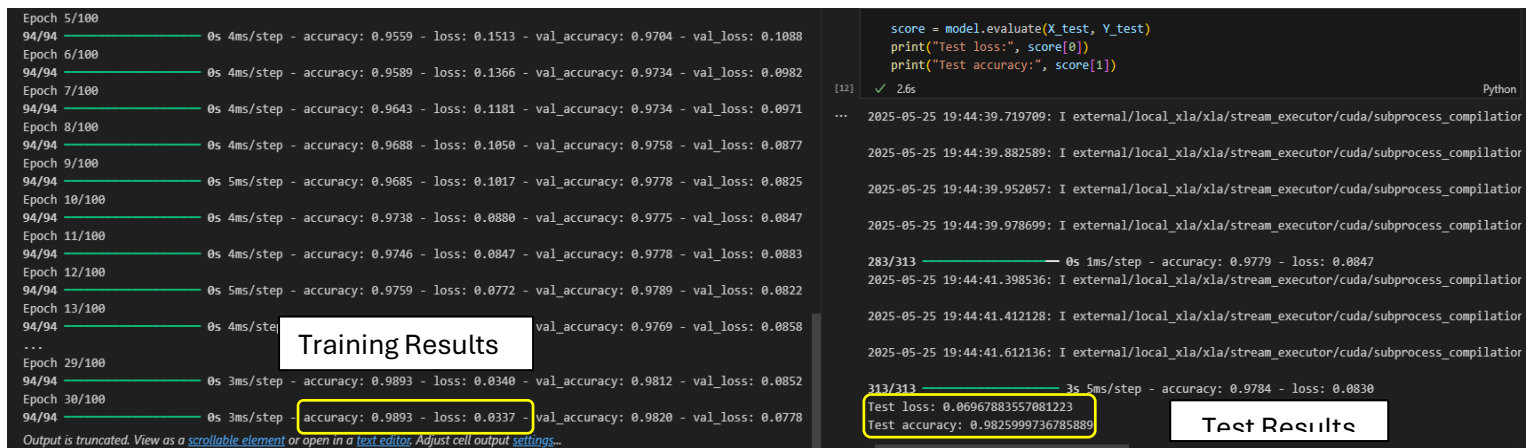


Figure 3: NN model training and test results

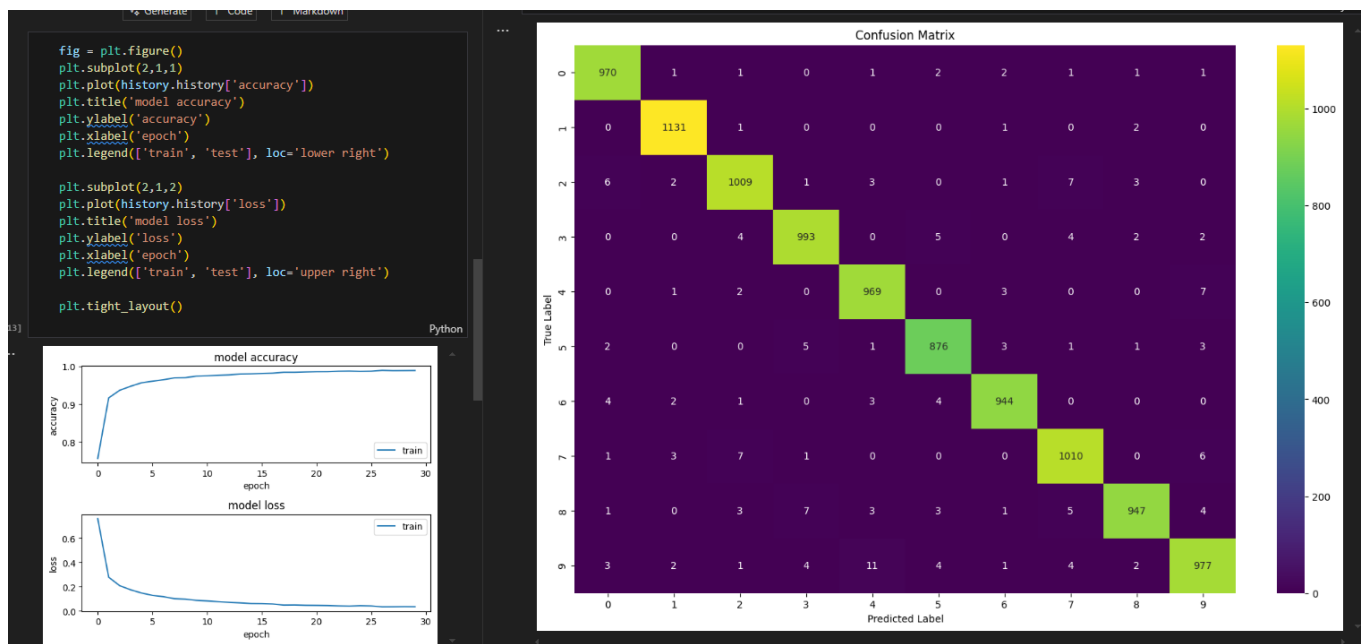


Figure 4: NN model plots for accuracy and loss over epochs (left) and confusion matrix (right)

Part B: Using CNN

Figure 5 below shows the model architecture used for the CNN model used for digit classification of the MNIST dataset. The CNN model has 2 convolution blocks using Conv2D layer with ReLu activation and maxpooling. Flatten layer converts the images to a 1D vector. Dropout layers are added to reduce overfitting. Model summary also shows the architecture (right side of the image).

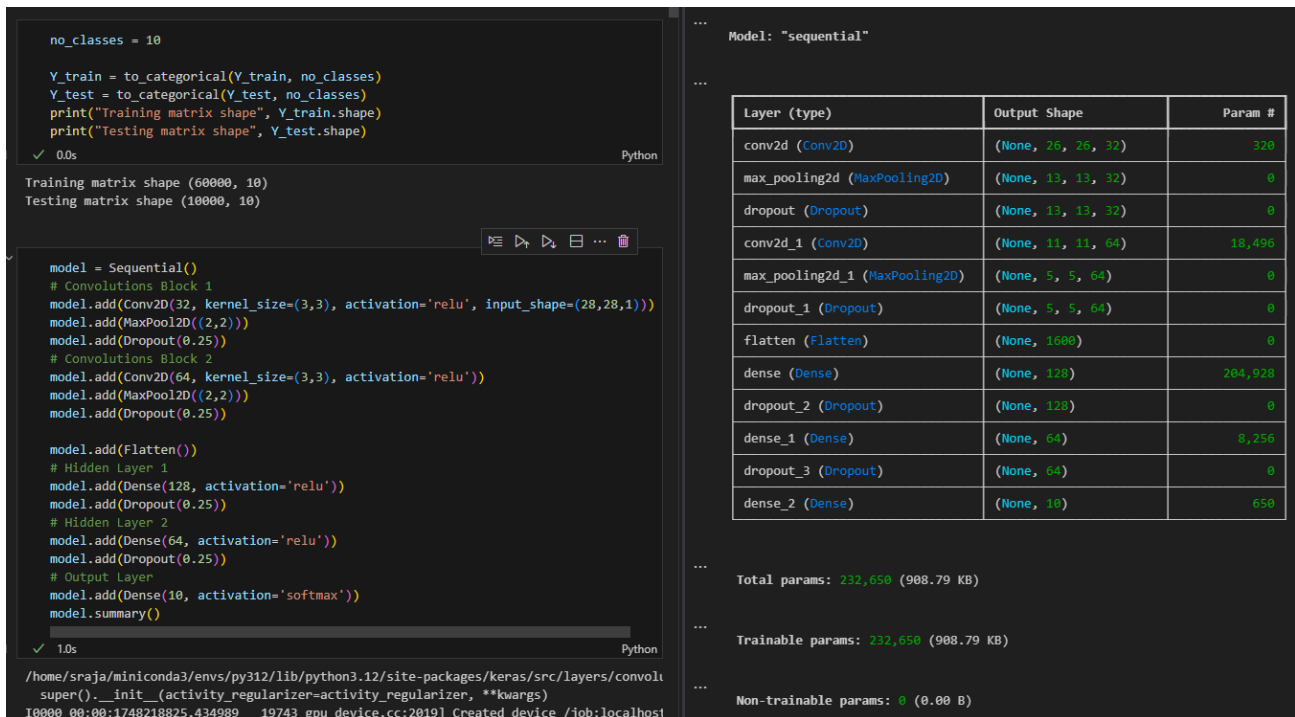


Figure 5: NN Model architecture for Digit Classification with MNIST

Figure 6 below shows the model configuration used for training. The model uses a batch size of 512 and Adam optimizer with learning rate of 1e-3 (0.001). Also, the model uses callbacks early stopping and model checkpoint. Figure 7 shows the accuracy and loss reported during training and test. The model achieves an accuracy of **98.26%** on the test dataset. Finally figure 8 shows the plots for training and validation accuracy and loss over epochs (left side) and the confusion matrix (right side). When comparing the results with NN model we see a slight improvement in accuracy using CNN model, CNN models are usually better suited for tasks that have spatial data.

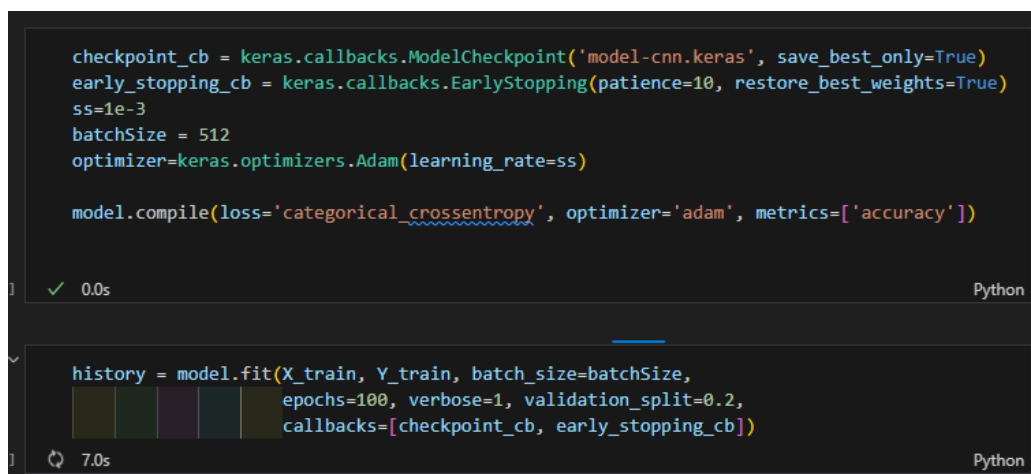


Figure 6: NN model configuration used for training

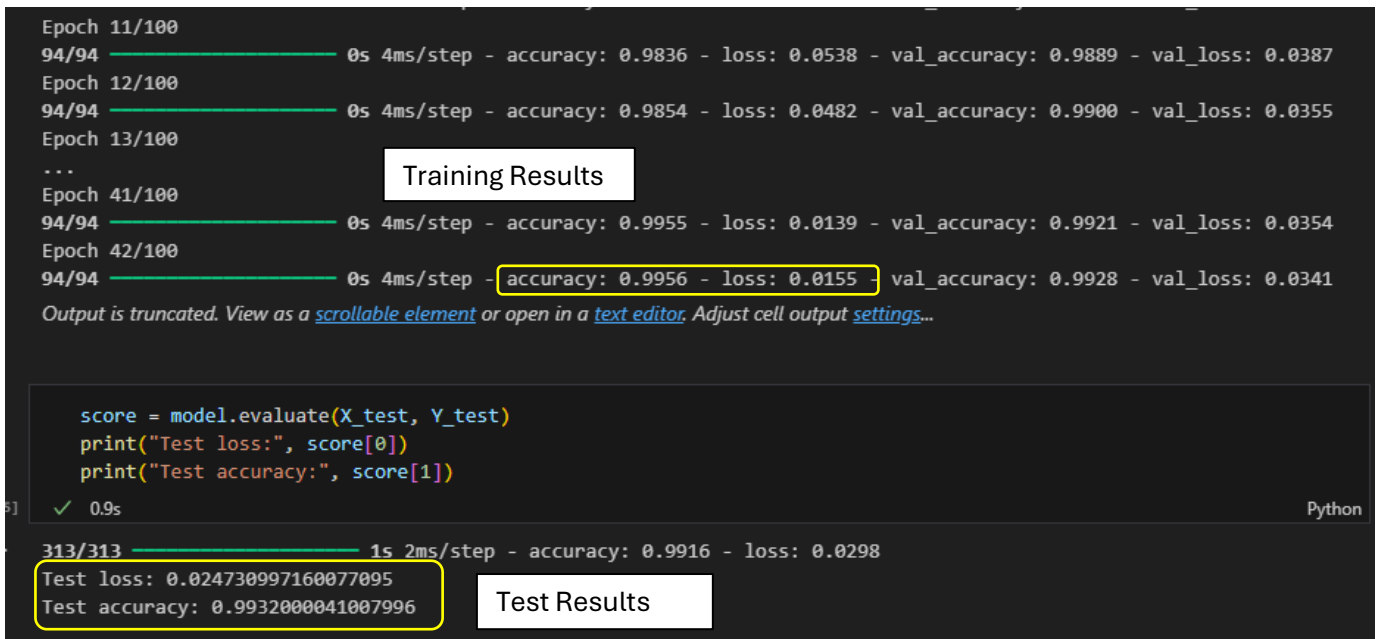


Figure 7: CNN model training and test results

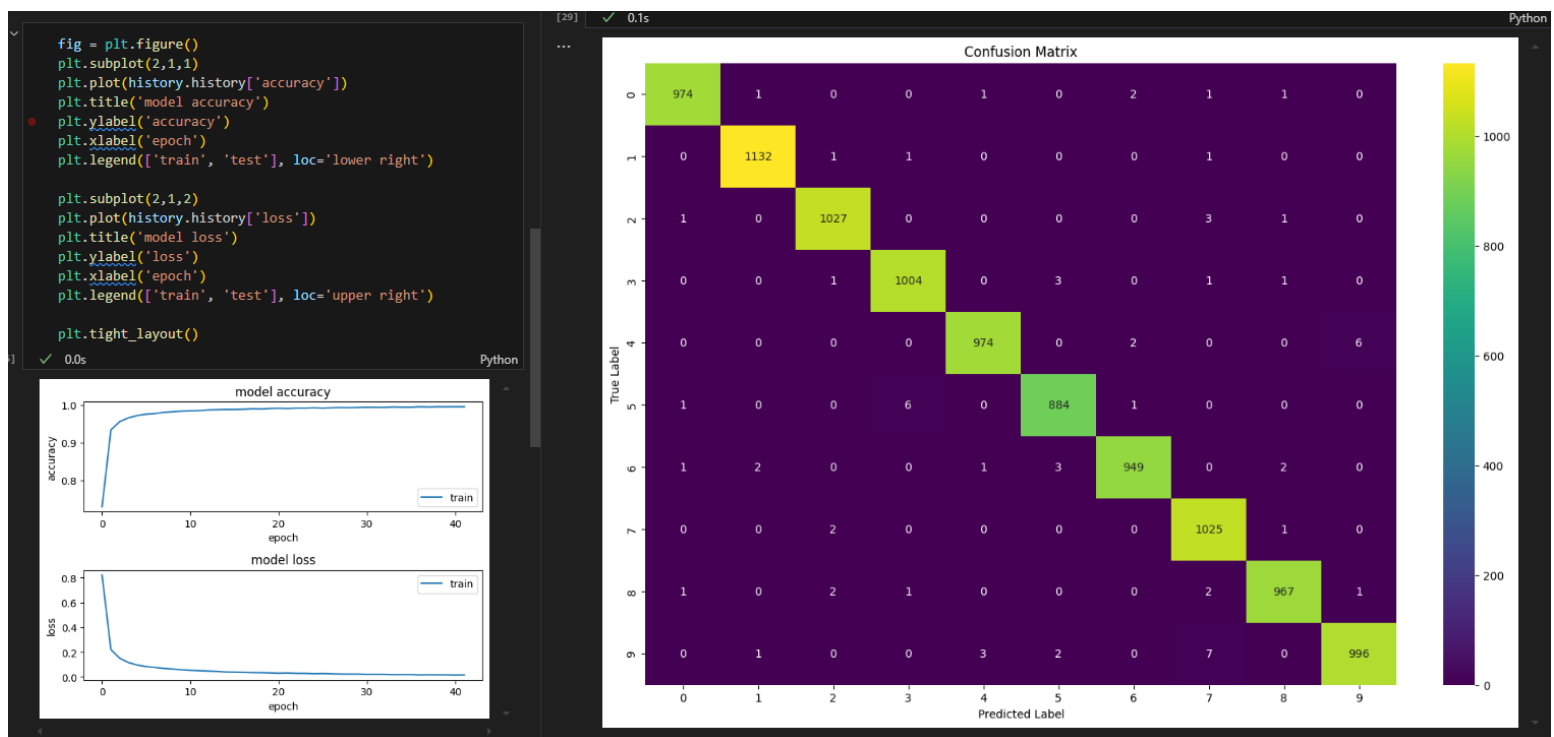


Figure 8: CNN model plots for accuracy and loss over epochs (left) and confusion matrix (right)

Question 2: Language Classification using RNNs

Figure 9 below shows the character-level RNN model created to predict the language of a given word. For this task we are using a dataset available from pytorch. For this model we have used PyTorch GRU model.

```

class charGRU(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, dropout_prob=0.3):
        super(charGRU, self).__init__()

        self.rnn = nn.GRU(input_size, hidden_size, bidirectional=True, dropout=dropout_prob)
        self.h2o = nn.Linear(hidden_size * 2, output_size)
        self.dropout = nn.Dropout(dropout_prob)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, line_tensor):
        rnn_out, hidden = self.rnn(line_tensor)
        final_hidden = torch.cat((hidden[0], hidden[1]), dim=1)
        dropped = self.dropout(final_hidden)
        output = self.h2o(dropped)
        output = self.softmax(output)

        return output

```

Python

```

n_hidden = 256
rnn = charGRU(n_letters, n_hidden, len(alldata.labels_uniq)).to(device)
print(rnn)

```

Python

```

charGRU(
  (rnn): GRU(58, 256, dropout=0.3, bidirectional=True)
  (h2o): Linear(in_features=512, out_features=18, bias=True)
  (dropout): Dropout(p=0.3, inplace=False)
  (softmax): LogSoftmax(dim=1)
)

```

Figure 9: RNN (GRU) Model architecture for Language Classification

Figure 10 below shows the model configuration used for training. The model uses a batch size of 64 and SGD optimizer with learning rate of $2e-1$ (0.2) and is trained for 50 epochs. Figure 11 shows the accuracy and loss reported during training and test, model achieves an accuracy of **82.00%** on the test dataset. Finally figure 12 shows the plots for training and validation accuracy and loss over epochs (left side) and the confusion matrix (right side).

```

def train(rnn, training_data, n_epoch=15, n_batch_size=64, report_every=1, learning_rate=0.5, criterion=nn.NLLLoss()):
    current_loss = 0
    all_losses = []
    rnn.train()
    optimizer = torch.optim.SGD(rnn.parameters(), lr=learning_rate)

    print(f"training on data set with n = {len(training_data)}")

    for epoch in range(1, n_epoch + 1):
        correct, total = 0, 0
        y_true, y_pred = [], []

        training_data = list(training_data)
        random.shuffle(training_data)
        batches = np.array_split(range(len(training_data)), len(training_data) // n_batch_size)

        for batch in batches:
            batch_loss = 0
            for i in batch:
                label_tensor, text_tensor, label, _ = training_data[i]
                label_tensor = label_tensor.to(device)
                text_tensor = text_tensor.to(device)
                output = rnn(text_tensor)
                loss = criterion(output, label_tensor)
                batch_loss += loss

            # Accuracy calculation
            _, pred_idx = output.topk(1)
            y_pred.append(pred_idx.item())
            y_true.append(label_tensor.item())

            batch_loss.backward()
            nn.utils.clip_grad_norm_(rnn.parameters(), 3)
            optimizer.step()
            optimizer.zero_grad()

        current_loss += batch_loss.item() / len(batch)

    epoch_loss = current_loss / len(batches)
    accuracy = accuracy_score(y_true, y_pred)
    all_losses.append(epoch_loss)
    if epoch % report_every == 0:
        print(f"Epoch {epoch} | Loss: {epoch_loss:.4f} | Accuracy: {accuracy*100:.2f}%")

    current_loss = 0

    return all_losses

```

Figure 10: RNN (GRU) Model architecture for Language Classification

```

start = time.time()
all_losses = train(rnn, train_set)
end = time.time()
print(f"training took {end-start}s")
✓ 5m 46.9s

training on data set with n = 17063
Epoch 1 | Loss: 1.3766 | Accuracy: 58.13%
Epoch 2 | Loss: 1.0219 | Accuracy: 68.09%
Epoch 3 | Loss: 0.8762 | Accuracy: 72.53%
Epoch 4 | Loss: 0.7883 | Accuracy: 75.54%
Epoch 5 | Loss: 0.7261 | Accuracy: 77.91%
Epoch 6 | Loss: 0.6672 | Accuracy: 79.44%
Epoch 7 | Loss: 0.6322 | Accuracy: 80.49%
Epoch 8 | Loss: 0.5883 | Accuracy: 81.67%
Epoch 9 | Loss: 0.5556 | Accuracy: 82.53%
Epoch 10 | Loss: 0.5149 | Accuracy: 83.78%
Epoch 11 | Loss: 0.4801 | Accuracy: 85.24%
Epoch 12 | Loss: 0.4498 | Accuracy: 85.63%
Epoch 13 | Loss: 0.4168 | Accuracy: 86.66%
Epoch 14 | Loss: 0.3977 | Accuracy: 87.24%
Epoch 15 | Loss: 0.3653 | Accuracy: 88.13%
training took 346.93567967414856s

evaluate(rnn, test_set, alldata.labels_uniq)
✓ 1.5s

Evaluation Accuracy: 82.00%

```

Figure 11: RNN (GRU) model training and test results

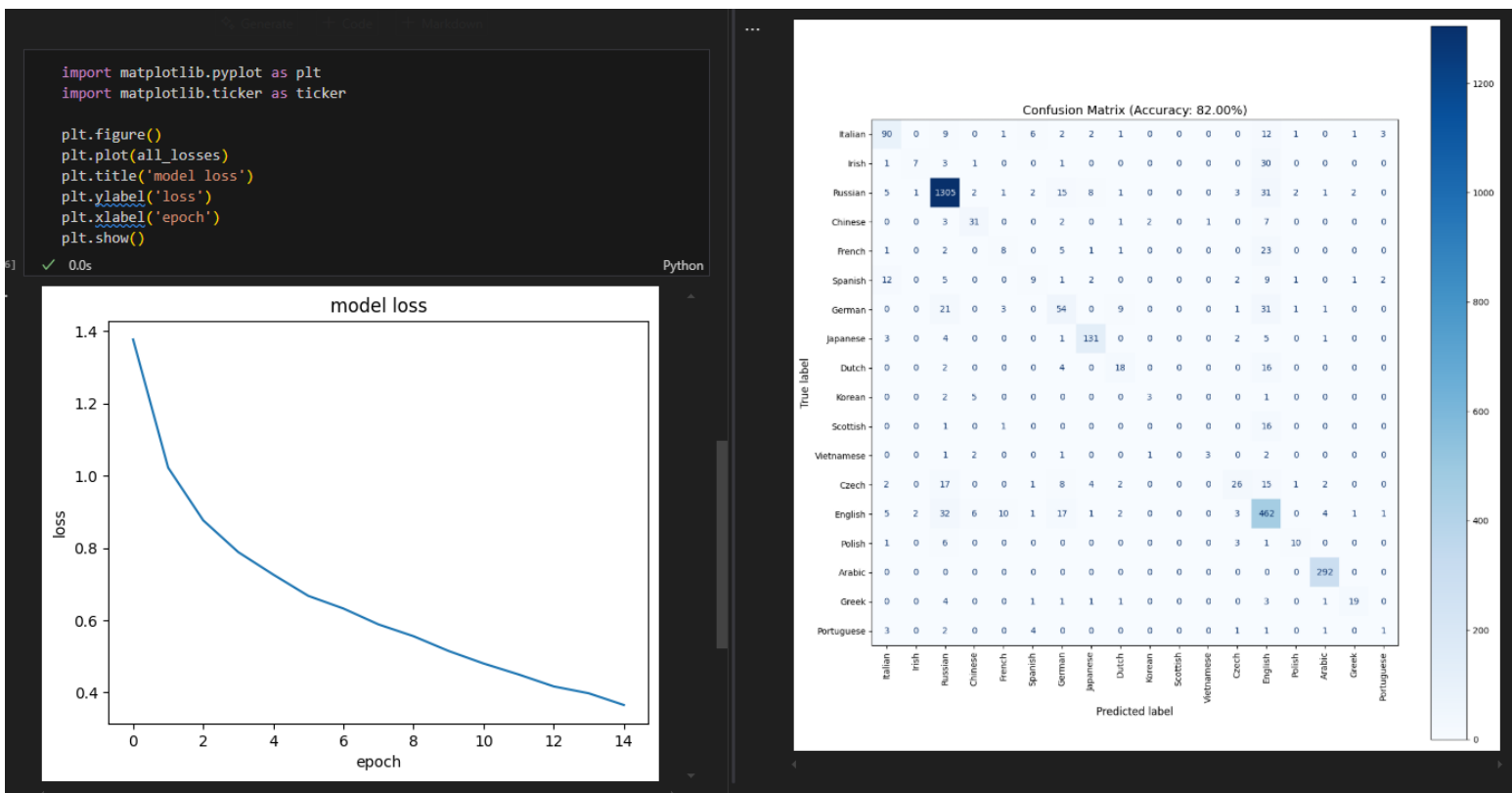


Figure 12: RNN (GRU) model plots for accuracy and loss over epochs (left) and confusion matrix (right)